

Case study: this document describes one specific deployment of the engine — the 2GuysTek YouTube channel — to make the moving parts concrete. The engine itself is generic: brand, niche, peers, subreddits, and feeds are all configured in `channel.yaml`. References to "2GT" / "2GuysTek" below are illustrative, not hardcoded.

2 G U Y S T E K

An AI research team that never sleeps and never bills.

It reads thousands of homelab, self-hosting, AI, and virtualization posts every week. It groups them into trends, weighs what's worth saying, and hands Rich a fresh content brief every Monday morning.

5 DATA STREAMS

7 SPECIALIST ROLES

\$0 PER RUN

RUNS ON ONE MAC

IN ONE PARAGRAPH

Every Sunday at 3 AM, a small program wakes up on Rich's Mac. It scans Reddit, YouTube, Hacker News, a curated list of vendor blogs, and the latest software releases relevant to homelabbers. Then a team of specialized AI workers — each with one job — summarize what they found, find the patterns, decide which patterns matter for 2GuysTek's audience, and write a draft brief: "here are five videos we could make this week, why each one matters right now, what to title each, and what to read first." Rich opens a private webpage Monday morning, swipes through the ideas, marks the ones he wants to make, and goes to work.

WHY THIS EXISTS

The information firehose

In any given week, ten thousand voices in homelab / self-hosting / AI / virtualization publish, post, debate, and ship new tools. New Proxmox release. Reddit thread about a security flaw. Lawrence Systems just covered Tailscale. A vendor blog announced end-of-life for a product half your audience uses.

No human can read all of it. Even spending an hour every morning, you miss the important things and waste time on noise. So Rich built — with help — an automated research team that does the reading *for* him, then hands him a short, prioritized list to react to.

Five data streams, autonomous

Five different kinds of signal, all pulled automatically — no person logs into anything. Each stream catches a different kind of story.



COMMUNITY PULSE

Reddit

The top posts from eight niche subreddits — r/homeLab, r/selfhosted, r/Proxmox, r/LocalLLaMA, and four more. This is where the audience argues, complains, and praises in their own words.



PEER SIGNAL

YouTube

Recent videos from 2GuysTek plus four peer channels — Tom Lawrence, Craft Computing, Jeff Geerling, ServeTheHome. So the system knows what other creators in the niche just covered (and what 2GT shouldn't redundantly chase).



AUTHORITATIVE NEWS

Vendor blogs & tech press

Official news from the companies behind the tools — XCP-ng, Vates, Veeam, Tailscale, Ubiquiti, Canonical — plus Phoronix, The Register, and Ars Technica IT. This is where you find out about a release *before* Reddit gets mad about it.



TECHNICAL ZEITGEIST

Hacker News

The biggest tech forum on the internet, filtered for stories that match 2GT's niche. High-engagement stories here often predict what'll be on Reddit in a day or two.



RELEASE PULSE

GitHub releases

New versions of seventeen open-source tools the 2GT audience actually runs — Ollama, llama.cpp, Vaultwarden, Immich, Home Assistant, Tailscale, Nextcloud, Pi-hole, OPNsense, and more. The instant a new version ships, the engine knows.

LIVE COUNT

377 signals in the database right now

reddit 161 youtube 86 blog 68 hackernews 32 github 30

The assembly line

Once a week, the engine walks every collected post through this assembly line. Each stop is a specialized AI worker with one job.

1

Summarize each item

~10 SECONDS EACH

For every Reddit post, YouTube title, blog entry, HN story, and GitHub release: an AI worker reads it and writes a one-sentence summary + a few key points + topic tags + a sentiment score (positive / neutral / negative) + a freshness score (is this brand new, or has the internet been debating it for years).



2

Group related items into trends

MATH, NOT MAGIC

Many summaries are about the same underlying story. A Proxmox kernel bug shows up in a Reddit thread, a Lawrence Systems video, a Phoronix article, and a forum post. The engine measures the similarity between all 377 summaries and clusters the related ones into *trends*. The Proxmox kernel cluster gets stronger as more sources mention it — that's signal that it's worth talking about.



3

Pick the hottest, most diverse trends

HEAT × NOVELTY × RECENCY

From dozens of trends, pick the most worth covering. Hotness = how many sources are talking about it, how new the story is, and how recently it broke. **And it enforces topic diversity:** it won't hand back five AI ideas in a row. One from homelab, one from security, one from virtualization, one from AI, one from networking. Audience gets variety.



4

Turn trends into actual video ideas

THE STRATEGIST

For each chosen trend, the strategist AI writes a sharp video angle — not "Top 10 Proxmox Tips" but *"Why I downgraded Proxmox's kernel after Tailscale broke — and why you might need to too"*. It explains why now, estimates audience fit, drafts 3–5 candidate titles, sketches thumbnail ideas, and flags any divisive risks (AI topics and Proxmox criticism both get auto-tagged because the audience is opinionated about both).



5

Draft a script outline + sources to verify

THE OUTLINE WRITER

For the top three ideas, a second AI drafts an actual outline: a cold-open hook, five to seven sections with timing estimates, beat-by-beat talking points, B-roll suggestions, a closing call-to-action, and — crucially — a list of links Rich should read or watch *before* writing the script, plus an "open questions" list of things to verify. The outline writer is trained on Rich's own past scripts so the voice and rhythm sound like him.



6

Score the titles & verify every citation

THE FACT CHECKER

Each candidate title gets scored 0-100 on click-worthiness based on what's worked for 2GT before. A separate worker then double-checks every URL the outline recommends as a source — if a link wasn't in the original signals (i.e. the AI made it up), it gets flagged as "unverified, check before quoting." This catches AI hallucination before it ever reaches a script.



7

Render a beautiful brief

DELIVERED

Everything gets stitched into a polished web page Rich opens on Monday morning. Five ideas, each with the angle, why now, ranked titles, thumbnail concepts, cross-post copy for Instagram/LinkedIn/Discord, a full outline for the top three, and the sources to read. Total time from "wake up at 3 AM" to "brief ready": about 10 minutes.

THE TEAM

Specialists, not generalists

No single AI does the whole job. Instead, seven specialists each do one thing well. Smaller and faster for easy tasks; bigger and slower only where it counts.



The Reader

Reads every incoming post. Produces a one-sentence summary, key points, topics, and sentiment for each. Fast and cheap because it does it 200 times a week.

Gemma 4 (8B) · ~10 sec each



The Cataloger

Doesn't read — it *compares*. Converts every summary into a mathematical fingerprint and finds which ones are about the same underlying story.

embeddings · < 1 sec each



The Labeler

Looks at each cluster and gives it a short, descriptive name plus picks the most evocative quote from inside it. Helps Rich understand at a glance what the trend is.

Gemma 4 (2B) · ~5 sec each



The Strategist

The lead creative. Looks at a trend and asks "what's the 2GT angle here?" Knows the channel's voice, what's worked before, and what topics are divisive. Writes the actual idea pitch.

Qwen 3 (30B) · ~12 sec per idea



The Outline Writer

For each green-lit idea, drafts a full beat-by-beat script outline including the cold open, B-roll, CTA, and a list of things to research first. Trained on the cadence of Rich's own scripts.

Qwen 3 (30B) · ~30 sec each



The Editor

Rewrites the prose passes from the Strategist and Outline Writer for fluency. Catches doubled words, awkward phrasing, and the small artifacts that small AI models sometimes produce.

Qwen 3 (30B) · ~2 sec per field



The Fact Checker

Not an AI — just plain logic. Goes through every URL the Outline Writer cites and confirms it was actually in the original signals. Anything else gets flagged with a visible "unverified" warning. AI models hallucinate citations; this catches it.

deterministic code · < 1 sec

WHERE THE AI LIVES

On Rich's desk. Not in the cloud.

Everything you've read about so far runs on one Mac sitting in Rich's office. The AI models — Google's Gemma and Alibaba's Qwen — are open-source and live entirely on the local hard drive. None of the data ever leaves the building.

That matters for three reasons. **Cost:** every weekly run costs zero dollars. **Privacy:** 2GT's analytics, scripts, and editorial decisions stay private. **Speed:** no waiting on someone else's API.

HARDWARE

Machine	Apple M5 Max
Memory	48 GB unified
AI models loaded	Gemma 4, Qwen 3
Total storage used	~50 GB
Cost per weekly run	\$0.00
Internet required	only to fetch data

RICH'S MONDAY MORNING

The human in the loop

The engine doesn't *decide* anything. It *recommends*. Rich makes every final call.

A

Open the dashboard

A private webpage on his Mac shows the latest brief at the top. He sees five ranked ideas, the three with full outlines, a line on what worked recently on the channel.

B

Swipe through

For each idea: ✓ Producing / 📄 Park / ✗ Reject. Optional reason. The engine remembers and won't pitch the same angle again unless something significant happens.

C

Track produced videos

When a green-lit idea becomes a real video, Rich pastes the YouTube ID into the dashboard. The engine then tracks how well it performs — and uses that to make future recommendations sharper.

- **The feedback loop**

Every fate Rich marks teaches the engine. Every video he produces gets its views, watch-time, and engagement fed back in. Over weeks and months, the engine's recommendations get more aligned with what actually works for 2GuysTek's audience — without ever losing Rich's voice or editorial judgment.

WHERE IT IS TODAY

By the numbers

377

SIGNALS IN THE DATABASE

5

INDEPENDENT DATA STREAMS

10

COMPLETED PIPELINE RUNS

52

IDEAS GENERATED TO DATE

~10

MINUTES PER WEEKLY RUN

7

SPECIALIST AI WORKERS

\$0

COST PER RUN, EVERY RUN

0

DATA SENT TO OUTSIDE AI SERVICES

AN IMPORTANT POINT

This doesn't write videos. It does the homework.

The engine never makes scripts. It never makes thumbnails. It never publishes anything.

What it does is read everything Rich would *want* to read if he had unlimited time, find the threads worth pulling on, and hand him a starting point. Every angle, title, outline, and source is a draft — Rich rewrites, rearranges, kills, or replaces anything that doesn't fit.

Think of it less like an AI ghostwriter and more like having a research team that works overnight: you get a stack of folders Monday morning, each with a topic, why it matters, and the homework already done. You're still the one on camera. You're still the one whose taste matters.

Critically, every AI-generated citation gets a verification flag. If the engine made something up, you see "⚠ unverified" before you read it. No surprise hallucinations.

PART II

Engineering notes

Everything above is the executive summary. What follows is the technical version for readers who want to see the actual architecture: the pipeline orchestration, the agent contracts, the model routing decisions, the math, the data model, and the observability surface.

Module map

A Python package (`content_engine`) with five layers. Each layer has a clear contract with the next. The orchestrator chains them in a fixed weekly DAG.

```

content_engine/
├── collectors/           ── inbound I/O, one module per source
│   ├── reddit.py        → reddit.com/r/X/top.json (no auth)
│   ├── youtube.py       → Composio→YOUTUBE_LIST_CHANNEL_VIDEOS
│   ├── feeds.py         → feedparser over RSS/Atom (data/feeds.yaml)
│   ├── hackernews.py    → hn.algolia.com/api/v1/search_by_date
│   ├── github_releases.py → api.github.com/repos/<repo>/releases
│   └── composio_client.py → auto-attaches connected_account by toolkit
├── processors/          ── transform raw signals into structured state
│   ├── summarize.py     → per-signal Pydantic-validated extraction
│   └── cluster.py       → embed → cosine → agglomerative → label
├── synthesizers/        ── generate the actual brief
│   ├── ideas.py         → topic-diversified IdeaCandidate per cluster
│   ├── outlines.py      → ScriptOutline per top-N idea
│   ├── editor.py        → unconstrained-mode prose polish + artifact regex
│   ├── citation_verifier.py → URL whitelist check (deterministic, no LLM)
│   ├── title_scorer.py  → vidiq OR heuristic CTR scoring per title
│   └── voice.py         → one-shot voice-guide extraction from scripts
├── reports/             ── styled HTML render
│   └── render.py        → Jinja2 → reports/weekly_<cycle>.html
├── web/                 ── FastAPI feedback UI (separate process)
│   └── app.py           → dashboard, swipe triage, run trigger
├── eval.py              ── golden test harness (tests/golden/)
├── ollama_client.py     ── HTTP harness: structured output + escalation
├── pipeline.py          ── orchestrator: collect→summarize→cluster→ideate→outline→rende
├── schemas.py           ── Pydantic contracts for every agent I/O
├── db.py                ── SQLite layer (WAL, 8 tables)
├── config.py            ── Settings (frozen dataclass), ENV via dotenv
└── cli.py               ── Typer entry: collect / run / gui / eval / polish / ...

```

LANGUAGE

Python 3.14

STORAGE

SQLite (WAL) + LanceDB

INFERENCE

Ollama (local)

MODEL ROUTING

Five tiers, one philosophy

Use the smallest model that can do the job, escalate on low confidence or schema-validation failure. Embedding is its own dedicated model. All running locally on Apple Silicon via Ollama (Metal backend).

TIER	MODEL	PARAMS	QUANT	DISK	USED FOR
embed	nomic-embed-text	137M	F16	274 MB	summary → vector for cluster dedupe
fast	gemma4:e2b	~2B effective	Q4_K_M	~2 GB	cluster labels, triage classification
mid	gemma4:e4b	~8B	Q4_K_M	9.6 GB	per-signal structured summarization
heavy	qwen3:30b-a3b-instruct-2507	30B MoE (3B active)	Q8_0	33 GB	idea + outline synthesis (JSON mode)
polish	qwen3:30b-a3b-instruct-2507	30B MoE (3B active)	Q8_0	(shared)	unconstrained prose rewrite

WHY MOE FOR HEAVY

Qwen 3 30B-A3B is a 30B-parameter mixture-of-experts model with ~3B active per token. Inference cost scales with active params, so it runs roughly as fast as a 3B dense model on Apple Silicon while keeping 30B-class reasoning quality. Replaced dense Gemma 4 31B Q8 after a single Q8 call took 540 s vs 10 s on the MoE.

WHY A SEPARATE POLISH TIER

Structured-output (Ollama `format=schema`) calls bake JSON tokens into every decoding step, which the smaller Q4 models do well enough but produce occasional artifacts (doubled words, leaked syntax). The polish pass re-runs the prose-only fields in *unconstrained* mode — Q8 MoE, free-form generation — and merges back into the schema.

Seven workers, hard contracts

Every agent has a Pydantic input/output contract validated on the way in *and* out. A parse failure is a signal to escalate, not to retry blindly. All telemetry (input hash, output, confidence, latency, errors, escalations) is logged to `agent_runs` for eval and regression detection.

summarize_signal

MID → HEAVY ON ESCALATION

processors/summarize.py

INPUT

RawSignal row

platform, title, url, body (truncated to 6KB), metrics, author, posted_at

OUTPUT SCHEMA

SignalSummary

- one_line (≤200 chars)
- key_points (1-5)
- topics (1-4 from TopicTag enum)
- sentiment (5-level enum)
- novelty (0-1)
- confidence (0-1)

STRATEGY

Ollama JSON mode with the schema enforced. Prompt is in `prompts/summarize_signal.txt`. Confidence < 0.55 triggers retry with the heavy tier. Run latency target: 8-12s on gemma4:e4b.

cluster_label

FAST TIER

processors/cluster.py

INPUT

cluster members + metrics

cluster_id, signal_ids, dominant_topics, avg_sentiment, heat_score, top 20 summary one-liners.

OUTPUT

`_ClusterLabel`

- label (≤120 chars, 3-7 words)
- representative_quote (verbatim from inputs)
- confidence (0-1)

STRATEGY

Cheap descriptive task — gemma4:e2b at ~5s each. Numeric metrics (heat, sentiment) are computed deterministically in Python first; the LLM only contributes the human-readable label + quote.

generate_idea

HEAVY TIER

FOLLOWED BY POLISH PASS

synthesizers/ideas.py

INPUT CONTEXT

- cluster label + heat + topics
- up to 25 summary one-liners
- extracted voice guide (~4KB)
- last 8 2GT videos w/ tier tag
- "what's working" / "underperforms"
- peer channels + prior ideas (90d)
- risk-flag rules (AI, Proxmox)

OUTPUT SCHEMA

IdeaCandidate

- angle (≤ 300 chars)
- why_now (≤ 400 chars)
- audience_fit
- format (short/quick_win/long_form/journey)
- suggested_titles (3-5)
- thumbnail_concepts (2-4)
- cross_post (IG/LI/Discord)
- risk_flags []
- fatigue_score (0-1)
- confidence (0-1)
- source_cluster_ids, source_signal_ids

SELECTION UPSTREAM

Diversification gate selects up to N clusters with *topic diversity*: round-robin across distinct primary topics with `max_per_topic=2`. Without this you'd get 5 AI ideas in a row whenever LocalLLaMA is on fire.

Prior-ideas list (90d window) is fed in to prevent re-pitching the same angle without significant new news.

generate_outline

HEAVY TIER

FOLLOWED BY POLISH PASS

synthesizers/outlines.py

INPUT CONTEXT

- full IdeaCandidate
- voice_guide.md (cadence + lexicon)
- cluster representative quote
- every source-signal title + URL + body excerpt
- risk flags from the idea

OUTPUT SCHEMA

ScriptOutline

- hook (≤ 600 chars, ~15s of read time)
- cold_open_seconds
- sections [3-12] of OutlineSection
- cta (≤ 240 chars)
- voice_notes (cadence reminders)
- sources [2-10] of ResearchSource
- open_questions []
- estimated_runtime_min
- confidence (0-1)

SELECTION

Outlines are expensive (~30s + ~2s polish). The pipeline only generates them for the top-3 highest-confidence ideas in a cycle. Open-questions field is non-optional — the prompt explicitly asks the model to *self-report* things it can't verify from the input signals, so Rich knows what to check.

editor (polish)

POLISH TIER · UNCONSTRAINED

`synthesizers/editor.py`

INPUT

JSON snippet containing only the prose fields of the parent agent's draft (e.g. `angle` / `why_now` / `audience_fit` for idea, `hook` / `cta` for outline). Field list is explicit so the polish pass can't touch structured numeric fields.

STRATEGY

- No `format=schema` constraint
- Lower temperature (0.2)
- Prompted as "editor cleaning up draft text"
- Falls back to original on JSON parse failure (never breaks pipeline)
- Heuristic artifact detector runs alongside for telemetry

WHAT IT CATCHES

- doubled words ("a a", "salvage salvage")
- weird hyphenation ("a-neutral")
- truncated mid-word
- leaked JSON syntax fragments
- stray control tokens
- common token-noise typos

score_titles

VIDIQ → HEURISTIC FALLBACK

synthesizers/title_scorer.py

VIDIQ PATH

If `COMPOSITION_API_KEY` is set and `vidiq` is connected, calls `VIDIQ_SCORE_TITLE` per title. 5 credits/call. Real CTR scoring against YouTube's ranker history.

HEURISTIC FALLBACK

Deterministic 0-100 score from regex levers, calibrated to 2GT's actual breakouts:

- + length sweet spot 35-65
- + numbers, year tags, parens
- + "vs" / comparison framing
- + contrarian verbs (stop, killed, broke, ditched)
- + first-person POV
- ALL CAPS, multiple !
- "ultimate guide" / "how to" / clickbait phrases

OUTPUT

Each idea's `title_scores` gets populated with [{title, score, source}] sorted desc. The report template highlights the top 2 in emerald. Source tag is preserved so we can tell vidiq scores from heuristic at a glance.

verify_citations

DETERMINISTIC · NO LLM

synthesizers/citation_verifier.py

WHY IT EXISTS

LLMs hallucinate plausible-looking URLs. In early runs the outline writer cited `github.com/2GuysTek/<made-up-repo>`. Untrustworthy citations corrode the value of the whole brief.

LOGIC

1. Normalize URL (lowercase host, strip trailing slash)
2. Build allowlist = source_signal URLs ∪ small infra-docs whitelist (`proxmox.com`, `github.com`, `kubernetes.io`, ...)
3. For each `ResearchSource`, set `verified = url ∈ allowlist`

SURFACE

Unverified sources render with amber border + "⚠ unverified" pill + "verify before quoting" note in the report. Per-cycle hallucination rate is logged to `agent_runs` for trend tracking (currently averaging ~8-14%).

How an agent call actually works

All five LLM-backed agents share a single entry point: `run_agent()` in `ollama_client.py`. The function is the enforcement layer — it owns structured output, validation, escalation, retry, and observability.

RUN_AGENT FLOW

- 1 Render prompt.** Load versioned template from `prompts/` (no Jinja in prompts on purpose — grep-friendly). Substitute `{{var}}`.
- 2 Call Ollama with the schema.** POST to `/api/chat` with `format = SchemaCls.model_json_schema()`. Ollama constrains decoding to produce valid JSON conforming to that schema.
- 3 Validate.** `Pydantic.model_validate_json()`. Catch `ValidationError` + `json.JSONDecodeError`.
- 4 Check confidence.** Extract `parsed.confidence`. If `< confidence_retry_below (0.55)` *and* a stronger tier exists, escalate.
- 5 Log to agent_runs.** `input_hash`, output JSON, confidence, elapsed_ms, escalated_from, error.
- 6 Return.** `(parsed_or_None, AgentResult)` tuple. Caller decides whether to persist.

ESCALATION ORDER

`fast → mid → heavy`

Triggered by either schema-validation failure or low confidence. The harness remembers `escalated_from` so we can see downstream which tier originally tried and failed.

RETRY ON TRANSIENT ERRORS

`tenacity` decorator on the underlying `httpx` call: 3 attempts, exponential backoff 1-8s, retries on `httpx.HTTPError` and `OllamaError` only (not on schema failures — those go through escalation instead).

HTTP TIMEOUT

900 s per call. Q8 unconstrained generation on dense models can take 5-9 min for a long outline; this is generous so we never lose work on a slow inference.

ENV NORMALIZATION

`OLLAMA_HOST` is normalized at startup — bare `host:port` gets `http://` prepended, bind addresses (`0.0.0.0`, `::`) get rewritten to `localhost`. Caught a real bug where Ollama's daemon plist set bare `host:port` for LAN binding.

THE CLUSTERING

From 377 summaries to a handful of trends

Clustering happens entirely in Python — no LLM is involved in the math. The LLM only names the resulting clusters.

STEPS

1. **Embed.** nomic-embed-text via Ollama /api/embed. Input is "{one_line} :: {' / '.join(key_points)}".
2. **L2-normalize** each vector for cosine distance.
3. **Agglomerative cluster** (sklearn): metric=cosine, linkage=average, distance_threshold=0.30.
4. **Per cluster:** top-3 dominant topics from member topic counts; mean sentiment in [-1, 1]; mean novelty.
5. **Heat score** from formula opposite.
6. **Label** via the cluster_label agent (fast tier) and persist to the clusters table.

HEAT SCORE FORMULA

```
recency = 1 / (1 + avg_age_hours / 48)
heat = n_signals × (0.4 + 0.6 × avg_novelty) × recency
```

Volume is the dominant factor (linear scaling). Novelty has a floor (0.4) so even a cluster about an old topic gets some weight if it's spiking in volume — but a novel topic gets nearly 2× the multiplier of a stale one. Recency decays with a ~2-day half-life so a Reddit thread from 6 days ago competes fairly against one from this morning.

Topic-diversification gate is applied *after* heat ranking — see `_select_diverse_clusters()` in `synthesizers/ideas.py`.

DATA LAYER

8 tables, SQLite + WAL

One file at `data/engine.db`. Schema lives in `db.py` — grep-friendly, easy to evolve. WAL mode so the GUI process can read while the pipeline writes.

TABLE	HOLDS	WRITTEN BY	NOTABLE INDEXES / CONSTRAINTS
signals	raw collected items	collectors/*	UNIQUE(platform, external_id)
summaries	structured per-signal output	processors/summarize	PK signal_id → signals(id)
clusters	named trend groups per cycle	processors/cluster	idx(cycle_id)
ideas	IdeaCandidate JSON + fate	synthesizers/ideas, web/app	PK idea_id; idx(cycle_id, fate)
outlines	ScriptOutline JSON	synthesizers/outlines, citation_verifier	PK idea_id → ideas(idea_id)
video_performance	YT analytics per produced video	web/app, future cron	PK video_id; FK idea_id
agent_runs	every LLM call (telemetry)	ollama_client.run_agent	idx(agent, created_at)
cycles	pipeline run metadata	pipeline, web/app	PK id (timestamp-based)

WHY SQLITE, NOT POSTGRES

Single-user local app. SQLite + WAL gives concurrent read/write with zero setup, zero daemon, zero cloud bill. The whole DB is portable as one file. If the engine ever moves to a real server, the schema in `db.py` is straightforward Postgres-compatible SQL.

WHY ALSO LANCEDB

Reserved for vector dedupe of past ideas (planned). Idea embeddings will be persisted so the synthesizer can do a similarity search before pitching anything — if a candidate is within ϵ of a 90-day-old idea, surface "you've already pitched this" unless there's a significant new event tied to it.

The weekly DAG

A single function in `pipeline.py` chains every step. No DAG framework, no message bus — for a single-user local app the overhead of Airflow / Prefect / Dagster isn't worth it.

```

new_cycle(notes=...)
|
▼
summarize_pending(limit=200)      — per-signal LLM, mid tier
|
▼
cluster_recent(days=7, threshold=0.30) — embed + sklearn + per-cluster label
|
▼
generate_for_clusters(top_n=6, max_per_topic=2)
|   ↳ generate_idea (heavy) → polish (polish) → IdeaCandidate
|
▼
score_cycle_ideas()              — vidiq + heuristic title scoring
|
▼
generate_for_ideas(only_top_n=3)
|   ↳ generate_outline (heavy) → polish (polish) → ScriptOutline
|
▼
verify_cycle()                   — citation verifier (deterministic)
|
▼
render_weekly() → reports/weekly_<id>.html
|
▼
finish_cycle(status='ok')

```

SCHEDULING

macOS LaunchAgent at `~/Library/LaunchAgents/com.2gt.contentengine.weekly.plist`. Sunday 3am. A second LaunchAgent keeps the FastAPI GUI process running with crash-restart (KeepAlive + 10s throttle).

AD-HOC RUNS

`P0ST /run` from the GUI forks a detached subprocess via `subprocess.Popen(start_new_session=True)`. Lock file at `data/runs/.manual_run.lock` tracks the PID. Zombie detection in `_process_alive()` via `ps -p <pid> -o state=`.

EVAL

`engine eval` runs every JSON case in `tests/golden/<agent>/` against the registered agent and checks the output against expectations (`min_confidence`, `must_include_topics`, `sentiment_in`, keyword presence). Catches model-swap regressions.

OBSERVABILITY VIA AGENT_RUNS

Every single LLM call writes a row to `agent_runs`. The report template reads from it to render the per-cycle "agent telemetry" table. Queries you can run by hand for forensics:

```
-- Which agent is slowest on the last cycle?
SELECT agent, model_tier, AVG(elapsed_ms) ms, COUNT(*) n
FROM agent_runs WHERE cycle_id = (SELECT id FROM cycles ORDER BY started_at DESC LIMIT 1)
GROUP BY agent, model_tier ORDER BY ms DESC;

-- Where are escalations happening?
SELECT agent, escalated_from, model_tier, COUNT(*) n
FROM agent_runs WHERE escalated_from IS NOT NULL GROUP BY 1,2,3;

-- Hallucination rate trend over the last 8 cycles
SELECT cycle_id, json_extract(output_json,'$.hallucination_rate') rate
FROM agent_runs WHERE agent='verify_citations' ORDER BY id DESC LIMIT 8;
```

Five collectors, five wire protocols

SOURCE	PROTOCOL	AUTH	RATE CONSIDERATIONS
reddit	public JSON (reddit.com/r/X/top.json)	none	~1 req/sec inter-sleep
youtube	Composio SDK → YOUTUBE_LIST_CHANNEL_VIDEOS	Google OAuth via Composio	YouTube API quota
blog	feedparser over RSS/Atom (per data/feeds.yaml)	none	0.4s per feed, tolerant of parse failures
hackernews	Algolia HN /search_by_date	none	17 keyword queries, ~0.3s spacing, in-memory dedupe
github	REST /repos/<owner>/<repo>/releases	unauth (60/hr/IP)	~0.5s per repo; skips drafts/pre-releases

COMPOSIO WRAPPER TRICK

collectors/composio_client.py indexes connected accounts by toolkit name on initialization. When `execute(slug, ...)` is called, it derives the toolkit from the slug prefix (e.g. `YOUTUBE_LIST_CHANNEL_VIDEOS` → `youtube`) and auto-attaches the right `connected_account`. Composio v3's SDK doesn't do this auto-resolution, and calls fail with `InvalidParams: connected_account cannot be None` without it. A `Noop` fallback class is returned when `COMPOSIO_API_KEY` is unset so the engine still runs end-to-end (just without OAuth'd sources).

COST MODEL

Why this costs \$0 per run

A typical weekly cycle makes ~120 LLM calls. At GPT-4o pricing, that's roughly \$0.50-\$1.50 per cycle depending on context size — \$25-75/year for the engine to run on its own.

Running locally on the M5 Max, the same 120 calls cost zero dollars (the hardware was bought regardless, electricity for the ~10 min of inference is rounding error). Over the project's lifetime that's a meaningful number — and more importantly, every editorial decision the engine makes stays on this Mac.

PER-CYCLE CALL BUDGET

embeddings	~1 batch (377 vectors)
summarize_signal	~20-40 (new only)
cluster_label	~50-80
generate_idea	5-6
generate_idea.polish	5-6
generate_outline	3
generate_outline.polish	3
total LLM	~95-140

2 G U Y S T E K

Built privately on local hardware. No subscriptions, no cloud bills, no data leaving the building.

Live snapshot — May 2026